

Tutorial Prático

CRUD de Produtos com Express, Bun e MySQL

Prof. Ivan Luiz Pedroso Pires
Desenvolvimento de Software para Web — UNEMAT

01 de junho de 2026

Resumo

Neste tutorial você vai construir uma aplicação **CRUD** (Create, Read, Update, Delete) completa de produtos. O **backend** usa **Express.js** rodando no **Bun**, com banco **MySQL** acessado pelo **mysql2**, organizado no padrão **Routes** e **Controllers**. O **frontend** é **HTML** puro com **JavaScript** usando **fetch**. Duração estimada: **2h30**.

Sumário

1	Pré-requisitos	2
2	Visão geral do projeto	2
3	Passo 1 — Criar o banco de dados	3
4	Passo 2 — Inicializar o backend com Bun	3
5	Passo 3 — Conexão com o MySQL	4
6	Passo 4 — Controller de Produtos	4
7	Passo 5 — Rotas de Produtos	6
8	Passo 6 — Servidor Express	6
9	Passo 7 — Testar a API	7
10	Passo 8 — HTML do frontend	7
11	Passo 9 — JavaScript do frontend	8
12	Passo 10 — CSS mínimo	10
13	Passo 11 — Rodar tudo junto	10
14	Resolução de problemas comuns	10
15	Desafio final	11
16	Checklist de entrega	11

1 Pré-requisitos

Antes de começar, confirme que você tem instalado:

- **Bun** (<https://bun.sh>) — runtime JavaScript
- **MySQL Server** rodando em localhost:3306
- **Editor de código** (VS Code recomendado)
- **Navegador** moderno (Chrome, Firefox ou Edge)
- Cliente HTTP opcional: **Insomnia** ou **Thunder Client**

Teste no terminal:

```
1 bun --version
2 mysql --version
```

Atenção

Se algum comando não for reconhecido, instale antes de continuar. Bun: `curl -fsSL https://bun.sh/install | bash`.

2 Visão geral do projeto

Vamos construir duas partes que se comunicam por HTTP:

Frontend (porta livre)	Backend (porta 3000)
index.html	server.js
style.css	routes/produtoRoutes.js
script.js (fetch)	controllers/produtoController.js
	config/database.js

Fluxo de uma requisição:

Browser → fetch() → Express → Route → Controller → MySQL

Estrutura final de pastas:

```
1 01-06-2025/
2 |-- backend/
3 |   |-- database.sql
4 |   |-- package.json
5 |   '-- src/
6 |       |-- config/database.js
7 |       |-- controllers/produtoController.js
8 |       |-- routes/produtoRoutes.js
9 |       '-- server.js
10 '-- frontend/
11 |   |-- index.html
12 |   |-- style.css
13 |   '-- script.js
```

3 Passo 1 — Criar o banco de dados

Acesse o MySQL e crie a tabela `produtos`.

```
1 CREATE DATABASE IF NOT EXISTS dsw_crud;
2 USE dsw_crud;
3
4 CREATE TABLE IF NOT EXISTS produtos (
5   id INT AUTO_INCREMENT PRIMARY KEY,
6   nome VARCHAR(100) NOT NULL,
7   preco DECIMAL(10,2) NOT NULL,
8   estoque INT NOT NULL DEFAULT 0,
9   criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP
10 );
11
12 INSERT INTO produtos (nome, preco, estoque) VALUES
13 ('Caneta Azul', 2.50, 100),
14 ('Caderno 96 folhas', 15.90, 50),
15 ('Borracha', 1.20, 200);
```

Listing 1: backend/database.sql

Execute o script:

```
1 mysql -u root -p < backend/database.sql
```

Valide criando a tabela:

```
1 USE dsw_crud;
2 SELECT * FROM produtos;
```

Dica

A coluna `id` é `AUTO_INCREMENT`: o MySQL gera o número sozinho. Nunca passe `id` no `INSERT`.

4 Passo 2 — Inicializar o backend com Bun

Crie as pastas e o `package.json`:

```
1 mkdir -p backend/src/{config,controllers,routes}
2 cd backend
3 bun init -y
```

Edite o `package.json` ficando assim:

```
1 {
2   "name": "backend",
3   "version": "1.0.0",
4   "type": "module",
5   "scripts": {
6     "dev": "bun --watch src/server.js",
7     "start": "bun src/server.js"
8   },
9   "dependencies": {
10    "cors": "^2.8.5",
11    "express": "^4.21.2",
12    "mysql2": "^3.11.5"
13  }
14 }
```

Listing 2: backend/package.json

Instale as dependências:

```
1 bun install
```

Atenção

O campo "type": "module" permite usar import/export em vez de require.

5 Passo 3 — Conexão com o MySQL

Crie um **pool de conexões** (mais eficiente que abrir/fechar a cada query).

```
1 import mysql from 'mysql2/promise';
2
3 const pool = mysql.createPool({
4   host: 'localhost',
5   port: 3306,
6   user: 'root',
7   password: '',
8   database: 'dsw_crud',
9   waitForConnections: true,
10  connectionLimit: 10,
11 });
12
13 export default pool;
```

Listing 3: backend/src/config/database.js

Dica

Use mysql2/promise: permite await nas queries em vez de callbacks aninhados.

6 Passo 4 — Controller de Produtos

O **controller** contém a lógica do CRUD. Cada função recebe (**req**, **res**) e responde JSON.

```
1 import db from '../config/database.js';
2
3 export const listar = async (req, res) => {
4   try {
5     const [rows] = await db.query(
6       'SELECT * FROM produtos ORDER BY id DESC'
7     );
8     res.json(rows);
9   } catch (error) {
10    res.status(500).json({ erro: error.message });
11   }
12 };
13
14 export const buscarPorId = async (req, res) => {
15   try {
16     const { id } = req.params;
17     const [rows] = await db.query(
18       'SELECT * FROM produtos WHERE id = ?', [id]
19     );
20     if (rows.length === 0) {
21       return res.status(404).json({ erro: 'Produto nao encontrado' });
22     }
23     res.json(rows[0]);
24   } catch (error) {
25     res.status(500).json({ erro: error.message });
26   }
27 }
```

27 };

Listing 4: backend/src/controllers/produtoController.js (parte 1)

```
1 export const criar = async (req, res) => {
2   try {
3     const { nome, preco, estoque } = req.body;
4     if (!nome || preco == null) {
5       return res.status(400).json({ erro: 'Nome e preco obrigatorios' });
6     }
7     const [result] = await db.query(
8       'INSERT INTO produtos (nome, preco, estoque) VALUES (?, ?, ?)',
9       [nome, preco, estoque ?? 0]
10    );
11    res.status(201).json({
12      id: result.insertId, nome, preco, estoque: estoque ?? 0
13    });
14  } catch (error) {
15    res.status(500).json({ erro: error.message });
16  }
17 };
18
19 export const atualizar = async (req, res) => {
20   try {
21     const { id } = req.params;
22     const { nome, preco, estoque } = req.body;
23     const [result] = await db.query(
24       'UPDATE produtos SET nome=?, preco=?, estoque=? WHERE id=?',
25       [nome, preco, estoque, id]
26     );
27     if (result.affectedRows === 0) {
28       return res.status(404).json({ erro: 'Produto nao encontrado' });
29     }
30     res.json({ id: Number(id), nome, preco, estoque });
31   } catch (error) {
32     res.status(500).json({ erro: error.message });
33   }
34 };
35
36 export const deletar = async (req, res) => {
37   try {
38     const { id } = req.params;
39     const [result] = await db.query(
40       'DELETE FROM produtos WHERE id = ?', [id]
41     );
42     if (result.affectedRows === 0) {
43       return res.status(404).json({ erro: 'Produto nao encontrado' });
44     }
45     res.json({ mensagem: 'Produto removido com sucesso' });
46   } catch (error) {
47     res.status(500).json({ erro: error.message });
48   }
49 };
```

Listing 5: produtoController.js (parte 2 -- escrita)

Atenção

Sempre use ? (prepared statements) para inserir variáveis nas queries. Concatenação direta abre brecha para SQL Injection.

7 Passo 5 — Rotas de Produtos

As **rotas** só declaram qual URL e método HTTP chamam qual função do controller.

```

1 import { Router } from 'express';
2 import {
3   listar,
4   buscarPorId,
5   criar,
6   atualizar,
7   deletar,
8 } from '../controllers/produtoController.js';
9
10 const router = Router();
11
12 router.get('/', listar);
13 router.get('/:id', buscarPorId);
14 router.post('/', criar);
15 router.put('/:id', atualizar);
16 router.delete('/:id', deletar);
17
18 export default router;
```

Listing 6: backend/src/routes/produtoRoutes.js

Mapa de endpoints (combinando o prefixo definido no `server.js`):

Método	URL	Função
GET	/produtos	listar todos
GET	/produtos/:id	buscar um
POST	/produtos	criar
PUT	/produtos/:id	atualizar
DELETE	/produtos/:id	deletar

8 Passo 6 — Servidor Express

O `server.js` amarra tudo: aplica middlewares, registra rotas, sobe o servidor.

```

1 import express from 'express';
2 import cors from 'cors';
3 import produtoRoutes from '../routes/produtoRoutes.js';
4
5 const app = express();
6
7 app.use(cors());
8 app.use(express.json());
9
10 app.use('/produtos', produtoRoutes);
11
12 app.get('/', (req, res) => {
13   res.json({ mensagem: 'API do CRUD de Produtos rodando' });
14 });
15
16 const PORT = 3000;
17 app.listen(PORT, () => {
18   console.log('Servidor rodando em http://localhost:${PORT}');
19 });
```

Listing 7: backend/src/server.js

Rode o servidor:

```
1 bun run dev
```

Você deve ver: **Servidor rodando em http://localhost:3000.**

Dica

Middlewares importam:

- `cors()`: libera chamadas do frontend (origem diferente).
- `express.json()`: faz o parse do corpo JSON das requisições.

A **ordem** é importante: middlewares antes das rotas.

9 Passo 7 — Testar a API

Antes de codar o frontend, valide a API. No Insomnia/Thunder Client:

1. **GET** `http://localhost:3000/produtos` → deve retornar a lista.
2. **POST** `http://localhost:3000/produtos` com corpo JSON:

```
1 { "nome": "Régua 30cm", "preco": 4.50, "estoque": 30 }
```

3. **PUT** `http://localhost:3000/produtos/1` com novos dados.
4. **DELETE** `http://localhost:3000/produtos/1`.

Pratique

Crie 3 produtos novos via POST e confirme que aparecem no GET. Atualize um deles e depois apague.

10 Passo 8 — HTML do frontend

```
1 <!DOCTYPE html>
2 <html lang="pt-BR">
3 <head>
4   <meta charset="UTF-8">
5   <title>CRUD de Produtos</title>
6   <link rel="stylesheet" href="style.css">
7 </head>
8 <body>
9   <h1>Cadastro de Produtos</h1>
10
11   <form id="form-produto">
12     <input type="hidden" id="produto-id">
13     <label>Nome: <input type="text" id="nome" required></label>
14     <label>Preco: <input type="number" id="preco" step="0.01" required></label>
15     <label>Estoque: <input type="number" id="estoque" value="0" required></label>
16     <div class="botoes">
17       <button type="submit" id="btn-salvar">Salvar</button>
18       <button type="button" id="btn-cancelar" hidden>Cancelar</button>
19     </div>
20   </form>
21
22   <h2>Lista de Produtos</h2>
23   <table id="tabela-produtos">
24     <thead>
```

```
25     <tr>
26       <th>ID</th><th>Nome</th><th>Preco</th>
27       <th>Estoque</th><th>Acoes</th>
28     </tr>
29   </thead>
30   <tbody></tbody>
31 </table>
32
33 <script src="script.js"></script>
34 </body>
35 </html>
```

Listing 8: frontend/index.html

11 Passo 9 — JavaScript do frontend

Usamos a Fetch API para conversar com o backend.

```
1  const API = 'http://localhost:3000/produtos';
2
3  const form = document.getElementById('form-produto');
4  const inputId = document.getElementById('produto-id');
5  const inputNome = document.getElementById('nome');
6  const inputPreco = document.getElementById('preco');
7  const inputEstoque = document.getElementById('estoque');
8  const btnCancelar = document.getElementById('btn-cancelar');
9  const tbody = document.querySelector('#tabela-produtos tbody');
10
11 function criarCelula(texto) {
12   const td = document.createElement('td');
13   td.textContent = texto;
14   return td;
15 }
16
17 function criarBotao(rotulo, classe, id) {
18   const botao = document.createElement('button');
19   botao.textContent = rotulo;
20   botao.className = classe;
21   botao.dataset.id = id;
22   return botao;
23 }
```

Listing 9: frontend/script.js (parte 1)

```
1  async function listarProdutos() {
2    const resposta = await fetch(API);
3    const produtos = await resposta.json();
4    tbody.replaceChildren();
5    produtos.forEach((p) => {
6      const tr = document.createElement('tr');
7      tr.append(
8        criarCelula(p.id),
9        criarCelula(p.nome),
10       criarCelula(`R$ ${Number(p.preco).toFixed(2)}`),
11       criarCelula(p.estoque),
12     );
13     const tdAcoes = document.createElement('td');
14     tdAcoes.append(
15       criarBotao('Editar', 'btn-editar', p.id),
16       criarBotao('Excluir', 'btn-excluir', p.id),
17     );
18     tr.append(tdAcoes);
19     tbody.append(tr);
20   });
21 }
```



```
20 });
21 }
22
23 async function salvarProduto(evento) {
24     evento.preventDefault();
25     const dados = {
26         nome: inputNome.value,
27         preco: parseFloat(inputPreco.value),
28         estoque: parseInt(inputEstoque.value, 10),
29     };
30     const id = inputId.value;
31     const url = id ? `${API}/${id}` : API;
32     const metodo = id ? 'PUT' : 'POST';
33     await fetch(url, {
34         method: metodo,
35         headers: { 'Content-Type': 'application/json' },
36         body: JSON.stringify(dados),
37     });
38     resetarFormulario();
39     listarProdutos();
40 }
41
42 async function editarProduto(id) {
43     const resposta = await fetch(`${API}/${id}`);
44     const produto = await resposta.json();
45     inputId.value = produto.id;
46     inputNome.value = produto.nome;
47     inputPreco.value = produto.preco;
48     inputEstoque.value = produto.estoque;
49     btnCancelar.hidden = false;
50 }
51
52 async function excluirProduto(id) {
53     if (!confirm('Deseja excluir este produto?')) return;
54     await fetch(`${API}/${id}`, { method: 'DELETE' });
55     listarProdutos();
56 }
57
58 function resetarFormulario() {
59     form.reset();
60     inputId.value = '';
61     inputEstoque.value = '0';
62     btnCancelar.hidden = true;
63 }
64
65 form.addEventListener('submit', salvarProduto);
66 btnCancelar.addEventListener('click', resetarFormulario);
67
68 tbody.addEventListener('click', (evento) => {
69     const id = evento.target.dataset.id;
70     if (!id) return;
71     if (evento.target.classList.contains('btn-editar')) editarProduto(id);
72     else if (evento.target.classList.contains('btn-excluir')) excluirProduto(id);
73 });
74
75 listarProdutos();
```

Listing 10: script.js (parte 2 -- CRUD)

Atenção

Use `textContent` e `createElement` ao invés de `innerHTML` para evitar **XSS** (cross-site scripting) quando exibir dados vindos da API.

12 Passo 10 — CSS mínimo

```
1 * { box-sizing: border-box; }
2 body {
3   font-family: system-ui, sans-serif;
4   max-width: 800px; margin: 2rem auto; padding: 0 1rem;
5   background: #f5f5f5; color: #222;
6 }
7 form, table {
8   background: #fff; border-radius: 8px;
9   box-shadow: 0 1px 3px rgba(0,0,0,.1);
10 }
11 form { padding: 1.5rem; margin-bottom: 2rem; }
12 form input { width: 100%; padding: .5rem; margin-top: .25rem; }
13 button { padding: .5rem 1rem; border: none; border-radius: 4px; cursor: pointer; }
14 #btn-salvar { background: #2563eb; color: #fff; }
15 .btn-editar { background: #f59e0b; color: #fff; }
16 .btn-excluir { background: #dc2626; color: #fff; }
17 table { width: 100%; border-collapse: collapse; }
18 th, td { padding: .75rem; text-align: left; border-bottom: 1px solid #eee; }
```

Listing 11: frontend/style.css (resumido)

13 Passo 11 — Rodar tudo junto

1. Em um terminal: `cd backend && bun run dev`.
2. Abra `frontend/index.html` no navegador (botão direito → *Open with Live Server* no VS Code é o ideal).
3. Cadastre, edite e exclua produtos. Veja a lista atualizar.

Pratique

Confira no DevTools (F12) → *Network*: cada ação dispara uma requisição diferente para a API.

14 Resolução de problemas comuns

ERR_CONNECTION_REFUSED Backend não está rodando. Volte ao terminal e suba com `bun run dev`.

CORS blocked Faltou `app.use(cors())` antes das rotas no `server.js`.

ER_ACCESS_DENIED Usuário/senha errados em `config/database.js`.

ER_BAD_DB_ERROR Banco `dsw_crud` não existe. Rode o `database.sql`.

Cannot read properties of undefined Esqueceu `express.json()` — `req.body` fica vazio.

Tabela não atualiza Verifique se a função `listarProdutos()` é chamada após cada POST/PUT/DELETE.

15 Desafio final

Pratique

Adicione um campo categoria (VARCHAR 50) ao produto. Para isso você precisa:

1. Alterar a tabela: `ALTER TABLE produtos ADD categoria VARCHAR(50);`
2. Atualizar as queries de `INSERT` e `UPDATE` no controller.
3. Adicionar o campo no formulário HTML.
4. Adicionar o campo no objeto `dados` do `salvarProduto`.
5. Adicionar a coluna na tabela do frontend.

Repare como uma única feature toca **quatro camadas**: banco, controller, HTML e JS.

16 Checklist de entrega

- ☐ Banco `dsw_crud` criado com a tabela `produtos`.
- ☐ Backend rodando em `localhost:3000` sem erros.
- ☐ Endpoint `GET /produtos` retorna JSON.
- ☐ Frontend lista, cria, edita e exclui produtos.
- ☐ Estrutura `routes/` + `controllers/` + `config/` respeitada.
- ☐ Campo `categoria` (desafio) implementado.

Bom código! — Prof. Ivan Pires
ivan.pires@unemat.br — DSW 2026.1